HACKATHON · GUÍA DE PARTICIPANTES

Alenia AI Ticketing Challenge

15–16 octubre 2026 · Espacio Vizcaya, Madrid

EL RETO

Durante 48 horas construiréis agentes de IA capaces de analizar y resolver tickets de soporte de datos inspirados en operaciones reales de Ayvens — una de las mayores empresas de gestión de flotas de Europa. El stack es el mismo que se usa en producción: **LangChain, Databricks y OpenProject**.

Tenéis tres casos de uso para elegir. Podéis abordar uno, dos o los tres: **cuantos más casos resolvéis, más puntos podéis conseguir.** No hay obligación de cubrirlos todos, pero el equipo que lo haga partirá con ventaja.

CALENDARIO

Día / Hora	Qué pasa	Dónde
Jue 15 oct 09:30	**Checkpoint técnico** — Verificación de accesos: comprobad que podéis entrar a Databricks y a vuestro proyecto de OpenProject antes del kick-off. El equipo técnico estará disponible para resolver problemas de acceso. El manual técnico de registro os llegará por separado.	Teams
Jue 15 oct 10:00	**Kick-off** — Presentación del reto y arranque oficial. Se asume que todos los accesos están verificados.	Teams
Jue 15 oct 13:00	**Sincronía** — Check de progreso, desbloqueo de dudas de workflow	Teams
Jue 15 oct 17:00	**Sincronía** — Estado del día, preguntas finales de la jornada	Teams
Vie 16 oct 09:30	**Sincronía matutina** — Última oportunidad para preguntas antes del cierre	Teams
Vie 16 oct 14:00	■ **FREEZE DE DESARROLLO** — La infra se apaga para desarrollo. Tened el agente en su estado final antes de esta hora. **No se aceptan cambios de código después de las 14h.**	—
Vie 16 oct 16:00	**Presentaciones + tickets en directo** — La infra vuelve a estar disponible. Durante cada presentación, lanzaremos tickets nuevos que vuestro agente tendrá que resolver en tiempo real delante del jurado. 25 min + 5 min de preguntas.	Espacio Vizcaya

El equipo técnico de Alenia estará disponible en Teams durante ambas jornadas para resolver dudas de workflow e infraestructura — no sobre la solución que construís, eso es cosa vuestra.

LOS CASOS DE USO

Los tres casos de uso son retos reales de las operaciones de datos de Ayvens, modelados con datos sintéticos. Elegid los que queráis abordar.

UC	Nombre y descripción	Impacto
UC1	**Normalization Management** Validar y mantener mappings de normalización usados por Datamarts y reporting regulatorio.	Alto
UC2	**Monthly Report Reconciliation** Investigar discrepancias entre cifras reportadas mensualmente y las cifras del controlador de negocio.	Alto
UC3	**Data Verification & Correction** Detectar y resolver datos operativos faltantes, inconsistentes o incorrectos.	Alto

DETALLE DE LOS CASOS DE USO

UC1 — Normalization Management

Ayvens opera en más de 40 países. Cada subsidiaria utiliza sus propios códigos locales (tipo de contrato, estado, canal de venta...) que deben estar mapeados a un código holding global para que los Datamarts y el reporting regulatorio funcionen correctamente. Cuando aparece un código local nuevo, un valor cambia o el mapping es incorrecto, se abre un ticket.

— Aparece un código local nuevo sin mapping a holding

— Un valor holding existente cambia y rompe mappings downstream

— Valores incorrectos o sin mapear detectados en reporting

Flujo del agente:

1. Lee el ticket	read_ticket(ticket_id) — extrae subsidiaria, clase, código local afectado
2. Busca precedentes	retrieve_similar_tickets(query) — tickets resueltos similares como referencia
3. Consulta el mapping actual	MD_LOCALVALUE_T — verifica si el código local tiene HOLDINGID válido
4. Valida el valor holding	MD_HOLDINGVALUE_T — confirma que el holding de destino existe y está activo
5. Revisa el historial	MD_LOCALVALUEHISTORY_T — detecta cambios recientes que puedan haber roto el mapping
6. Solicita aprobación → comenta	create_comment() — análisis + propuesta de corrección del mapping
7. Solicita aprobación → cierra	update_status() — actualiza el estado del ticket

UC2 — Monthly Report Reconciliation

Cada mes, el equipo de data engineering publica un Fleet Report (capa Gold) que consolida datos de contratos y vehículos por subsidiaria. El controlador de negocio compara estas cifras con las suyas propias. Cualquier discrepancia — en número de contratos, estados, o actividad de negocio — genera un ticket de reconciliación.

— Diferencias en número de contratos o clientes entre el report y el controlador

— Discrepancias de estado (un contrato aparece activo en un sistema e inactivo en otro)

— Actividad de negocio faltante o duplicada en el report mensual

Flujo del agente:

1. Lee el ticket	read_ticket(ticket_id) — extrae período, subsidiaria, tipo de discrepancia
2. Busca precedentes	retrieve_similar_tickets(query) — patrones de reconciliación anteriores
3. Valida el Fleet Report	FLEET REPORT (Gold) — cifras reportadas para el período afectado
4. Compara con la fuente	CONTRACT (Silver/Bronze) — datos crudos de contratos y estados
5. Revisa el procesamiento	MAIN_T (monitoring) — detecta fallos de proceso o datos no llegados en esa fecha
6. Identifica la causa raíz	Determina si es error de datos, de proceso, de timing o de mapping
7. Solicita aprobación → comenta	create_comment() — causa raíz + responsable + propuesta de corrección
8. Solicita aprobación → cierra	update_status() — actualiza el estado del ticket

UC3 — Data Verification & Correction

Los datos operativos de contratos y vehículos llegan de fuentes locales de cada subsidiaria. A veces los registros llegan con campos faltantes, fechas incorrectas o datos de cliente o vehículo erróneos. Identificar el problema, localizar todos los registros afectados y determinar la corrección correcta requiere una investigación detallada a nivel de registro.

— Fechas faltantes o incorrectas en contratos o vehículos

— Datos de cliente o conductor ausentes o inconsistentes

— Código de vehículo, VIN o matrícula incorrectos

— Estado de contrato incorrecto respecto a la fuente

Flujo del agente:

1. Lee el ticket	read_ticket(ticket_id) — extrae registro afectado, tipo de error y subsidiaria
2. Busca precedentes	retrieve_similar_tickets(query) — correcciones similares del pasado
3. Valida el registro	CONTRACT / VEHICLE (Bronze/Silver) — verifica los campos reportados como incorrectos
4. Revisa el historial de carga	FILE LOG (monitoring) — rastrea el fichero fuente del registro afectado
5. Amplía el impacto	Busca otros registros con el mismo problema en la misma carga o subsidiaria
6. Determina la corrección	Establece el valor correcto y el camino de corrección en la fuente o en la capa Silver
7. Solicita aprobación → comenta	create_comment() — análisis completo + registros impactados + propuesta de fix
8. Solicita aprobación → cierra	update_status() — actualiza el estado del ticket

El entorno de Databricks sigue una arquitectura medallón: Bronze y Silver contienen datos operativos crudos y procesados; Gold contiene datos consolidados listos para reporting; las tablas SQL contienen los catálogos de normalización; y Monitoring registra la actividad de procesamiento. Todos los datos son sintéticos — generados estadísticamente a partir de patrones reales de Ayvens.

Tabla	Capa	Propósito	UC relevante
`CONTRACT`	Bronze / Silver	Datos de contratos de leasing por subsidiaria. Incluye fechas de entrega, inicio, cancelación y terminación; mileage contratado; valores residuales; estado y canal de venta.	**UC2, UC3**
`VEHICLE`	Bronze / Silver	Datos de vehículos: matrícula, VIN, marca, modelo, características técnicas (energía, CO2, puertas, asientos), precio de adquisición, kilometraje.	**UC3**
`FLEET REPORT`	Gold	Vista consolidada mensual: contrato + vehículo + cliente, con valores holding y locales de tipo de contrato, estado y canal de venta. Base del reporting al controlador de negocio.	**UC2**
`MD_LOCALVALUE_T`	SQL / Norm.	Catálogo de valores locales por subsidiaria y clase. PK: (LOCALCODE, CLASSID, SUBSIDIARYCODE). Cada registro mapea un código local a un HOLDINGID global.	**UC1**
`MD_HOLDINGVALUE_T`	SQL / Norm.	Catálogo de valores holding globales. PK: (HOLDINGID, CLASSID). Contiene nombre, descripción y jerarquía (PARENTID). Es la referencia maestra de normalización.	**UC1**
`MD_LOCALVALUE HISTORY_T`	SQL / Norm.	Historial de cambios en valores locales. Registra quién cambió qué, cuándo, y a qué HOLDINGID se mapeó. Clave para entender la raíz de un mapping roto.	**UC1**
`CF_SUBSIDIARIES_T`	SQL / Norm.	Referencia de subsidiarias: código, nombre de empresa, país. Útil para contextualizar tickets de un subsidiary específico.	**UC1, UC2, UC3**
`MAIN_T`	Monitoring	Log de ejecuciones de procesamiento: entidad, capa, total de filas procesadas, output y fecha. Permite detectar fallos o datos no llegados en una fecha concreta.	**UC2**
`FILE LOG`	Monitoring	Registro de ficheros fuente procesados: nombre de fichero, columnas y valores de PK, ruta y fecha de proceso. Permite trazar el origen exacto de un registro.	**UC3**

Las tablas están disponibles en vuestro Unity Catalog de Databricks. Las podéis consultar directamente desde vuestro notebook o agente usando SQL o la API de Databricks.

LA INFRAESTRUCTURA Y LAS HERRAMIENTAS

Lo que Alenia ha construido por vosotros

Vuestra misión es construir el agente LangChain que orquesta las herramientas. La capa de infraestructura ya está montada:

Databricks Serverless Workspace	Compute compartido, con un Unity Catalog aislado por equipo.

OpenProject	Un proyecto por equipo, acceso gestionado por service account. Aquí vivirán los tickets.
Modelo de lenguaje (LLM)	Endpoint compartido de Model Serving — Llama 3 / DeepSeek / Mistral. No necesitáis fine-tuning para el caso base.
Vector Search (RAG)	Índice Mosaic AI sobre tickets históricos sintéticos, expuesto como herramienta.
Gestión de secretos	Las credenciales están en el vault de Alenia — nunca en texto plano.

Contrato de herramientas — las funciones disponibles

Además de las 4 funciones del contrato, el agente puede generar y ejecutar queries SQL sobre las tablas del modelo de datos. Esta capacidad de **text-to-SQL** — transformar el texto del ticket en código SQL para consultar los datos relevantes — es una parte central del reto.

Función / Capacidad	Qué hace	Aprobación requerida
`read_ticket(ticket_id)`	Lee el contenido completo de un ticket de OpenProject.	No
`retrieve_similar_tickets(query)`	Busca tickets históricos similares usando RAG sobre el índice de Vector Search.	No
`SQL query (text-to-SQL)`	El agente genera SQL a partir del texto del ticket para consultar las tablas del modelo de datos. Debe presentar la query generada y su propósito antes de ejecutarla.	Sí ■
`create_comment(ticket_id, text)`	Añade un comentario a un ticket con el análisis y propuesta de resolución.	Sí ■
`update_status(ticket_id, status)`	Actualiza el estado de un ticket.	Sí ■

LAS REGLAS

■ Human-in-the-loop — obligatorio

Vuestro agente debe pedir aprobación explícita antes de dos tipos de acciones: (1) **Consultas SQL** — el agente traduce el texto del ticket a SQL (text-to-SQL) y debe presentar la query generada para aprobación antes de ejecutarla. (2) **Acciones de estado** — create_comment() y update_status() requieren aprobación antes de ejecutarse. Un agente que ejecuta queries o modifica estado sin aprobación no cumple los requisitos del reto. La calidad del mensaje de aprobación — cuánto contexto aporta para que el humano decida — forma parte de la puntuación.

■ Solo datos sintéticos

Todos los datos del entorno son sintéticos — generados estadísticamente a partir de patrones reales de Ayvens. No intentéis acceder a sistemas externos ni usar datos propios.

■ Solo las herramientas del contrato

El agente debe operar únicamente con las cuatro funciones del contrato. No podéis conectar APIs externas ni modificar la capa de infraestructura.

■ Entorno aislado por equipo

Vuestro Unity Catalog, vuestro proyecto de OpenProject y vuestras credenciales son exclusivos. No tenéis acceso a los entornos de otros equipos ni a sistemas de producción de Ayvens.

■ El código es vuestro

Podéis usar cualquier librería compatible con el entorno Databricks. LangChain es el punto de partida recomendado.

CRITERIOS DE PUNTUACIÓN

La puntuación máxima es de **135 puntos**: 100 de base más hasta 35 en bonificaciones. Conocer los criterios de antemano es parte del reto — construid teniendo esto en mente.

Puntuación base — 100 puntos

Criterio	Pts	Qué se evalúa
Routing y priorización	**20**	El primer paso — y el más importante. ¿El agente identifica correctamente el tipo de ticket, lo asigna al caso de uso adecuado y determina su prioridad antes de iniciar el análisis? Un routing incorrecto invalida todo lo que viene después.
Accuracy	**25**	Comparación del output de vuestro agente con el output ideal preparado por Alenia para cada ticket. Cuanto más se acerque vuestro análisis y propuesta de resolución a la solución de referencia, mejor nota.
Cost of outcome	**20**	Coste de compute y tokens para generar una resolución completa de un ticket. **Lo medimos nosotros desde la infraestructura** — no tenéis que calcularlo ni presentarlo. Un agente que resuelve bien con menos recursos puntúa más que uno que llega al mismo resultado consumiendo el doble.
Human-in-the-loop	**20**	El agente debe solicitar aprobación antes de dos tipos de acciones: las **queries SQL** que genera a partir del texto del ticket (text-to-SQL), y las **acciones de estado** (create_comment, update_status). Se evalúa tanto que se implemente correctamente como la calidad del mensaje de aprobación — ¿tiene suficiente contexto para que un humano decida de forma informada? Un "¿ejecuto esta query?" sin mostrar el SQL no es lo mismo que presentar la query, explicar qué busca y por qué.
Presentación y reflexión	**10**	Calidad de la demo, claridad de la explicación, y honestidad técnica sobre lo que no funciona y por qué. El jurado valora más a un equipo que sabe exactamente qué falló que uno que intenta disimularlo.
Calidad técnica	**5**	El agente no se rompe, respeta el contrato de herramientas, el código está estructurado y es legible. No se espera perfección, pero sí que sea mantenible.

Bonificaciones

Bonus	Pts	Condición

2 casos de uso	**+10**	Vuestro agente resuelve correctamente tickets de al menos 2 casos de uso distintos.
3 casos de uso	**+20**	Vuestro agente resuelve correctamente los 3 casos de uso. Este bono es acumulativo — sustituye al bono de 2 UCs (máximo +20, no +30).
UI / Dashboard	**+15**	Habéis construido una interfaz donde se puede ver el estado de los agentes en tiempo real y métricas básicas del proceso (tickets procesados, estado, coste, aprobaciones pendientes...).

Puntuación máxima teórica: 135 pts (100 base + 20 por 3 UCs + 15 por UI).

LA PRESENTACIÓN FINAL

El 16 de octubre a partir de las 16:00h en Espacio Vizcaya presentaréis vuestra solución al jurado. Cada equipo dispone de **25 minutos de presentación + 5 minutos de preguntas**.

Tickets en directo

Durante la presentación de cada equipo, lanzaremos tickets nuevos que vuestro agente tendrá que resolver en tiempo real delante del jurado. La infra vuelve a estar disponible a las 16:00h para esto. **Vuestro agente debe ser capaz de procesar inputs que no ha visto antes** — no es solo una demo de lo que ya funciona, es una prueba en vivo.

Estructura sugerida de los 25 minutos

— ¿Qué caso(s) de uso elegisteis y por qué? (contexto rápido, 2-3 min)

— Demo en vivo con tickets nuevos — el agente procesa, vosotros narráis lo que está haciendo

— El flujo de aprobación humana en acción — mostrad cómo el agente solicita confirmación y qué información aporta

— Métricas: accuracy conseguida, coste por ticket, número de UCs resueltos

— Qué funciona, qué no funciona, qué haríais diferente con más tiempo

Freeze de desarrollo a las 14:00h. A partir de esa hora no se aceptan cambios de código. Tened el agente estable y probado antes de ese momento.

CÓMO LLEGAR — PRESENTACIONES DEL 16 DE OCTUBRE

Dirección	Espacio Vizcaya · Calle Vizcaya 12, 28045, Madrid
Metro	Estación del Arte · Palos de la Frontera
Autobús	19 · 27 · 45
Parking	Garaje Hiomarsa (C. de Rafael de Riego, 14) · Parking Público Atocha (C. de Rafael de Riego, 5)
Presentaciones	A partir de las 16:00h — llegad antes de las 15:45h